WEEK 1 · DAY 1, PART 1 · OVERVIEW

Python for Fixed Income

Introduction, Part 1



Created by **Jeff Lenamon**

ABOUT 65 MIN WITH EXERCISES / 6 SECTIONS / 6 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk's Four Bonds

Put our book into Python. What does our first 1,000,000 face of QVNE pay a year, and what did it cost? What is our total face, and our price range? And clean up the broker's messy text.

ABOUT 65 MIN WITH EXERCISES / 6 SECTIONS / 6 EXERCISES

Your Route Through Day 1, Part 1

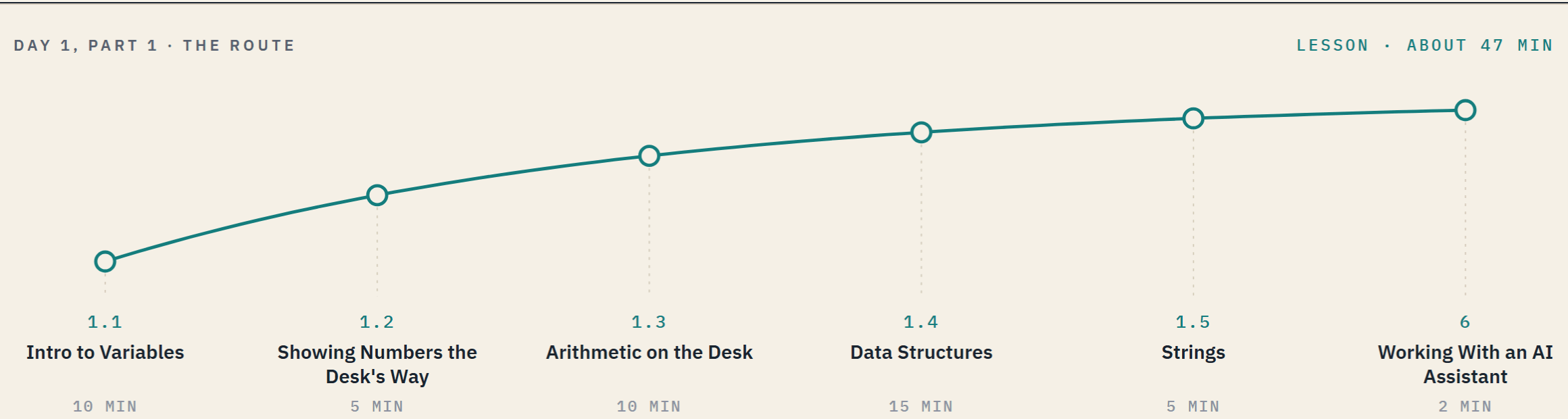


FIGURE 1 Six sections, in order, with about how long each takes.

Time. About 47 minutes for the lesson, then about 18 for the six exercises.

1.1 Intro to Variables

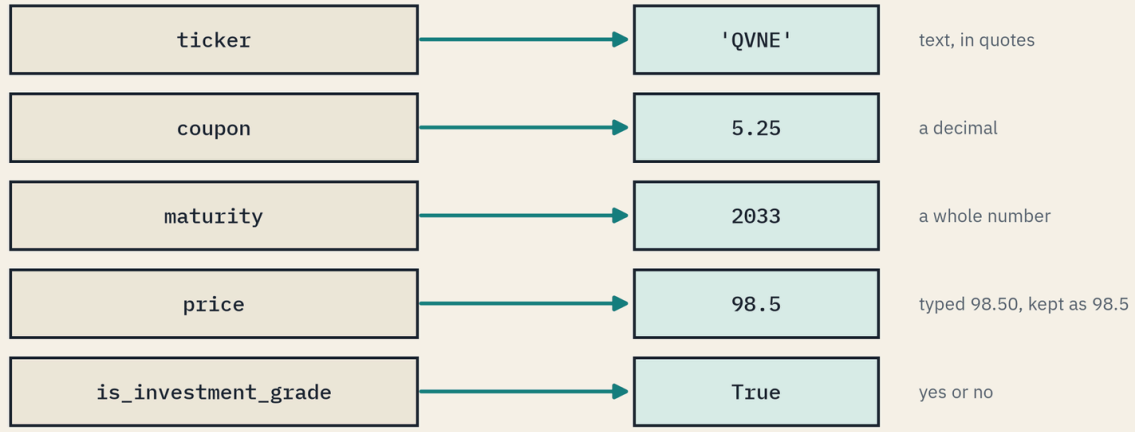


FIGURE 2 Five names, each pointing at its value.

STORE ONE BOND

```
ticker = 'QVNE'
coupon = 5.25      # percent
price = 98.50      # per 100 face
```

CHECK THE TYPES

```
print(type(ticker))
print(type(coupon))
print(type(maturity))
print(type(is_investment_grade))
```

```
<class 'str'>
<class 'float'>
<class 'int'>
<class 'bool'>
```

98.50 prints as 98.5: Python keeps the number, not the way it was typed.

1.2 Showing Numbers the Desk's Way

THE CODE

```
face = 1000000
annual_income = face * coupon / 100
print(f'QVNE pays ${annual_income} a year')
print(f'QVNE pays ${annual_income:,.2f} a year')
```

```
QVNE pays $52500.0 a year
QVNE pays $52,500.00 a year
```

MORE FORMAT CODES

```
print(f'Price: {price:.2f}')
print(f'Coupon: {coupon:.3f} percent')
```

```
Price: 98.50
Coupon: 5.250 percent
```

EXCEL SIDE BY SIDE

PYTHON	EXCEL	SHOWS
`f'{annual_income:,.2f}'`	`=TEXT(A1,"#,##0.00")`	52,500.00
`f'{price:.2f}'`	`=TEXT(A1,"0.00")`	98.50
`f'{0.0525:.2%}'`	`=TEXT(A1,"0.00%")`	5.25%
`f'{ticker} pays {coupon:.2f}'`	`=A1&" pays "&TEXT(B1,"0.00")`	QVNE pays 5.25

The note's first answer: 52,500.00 dollars a year.

1.3 Arithmetic on the Desk

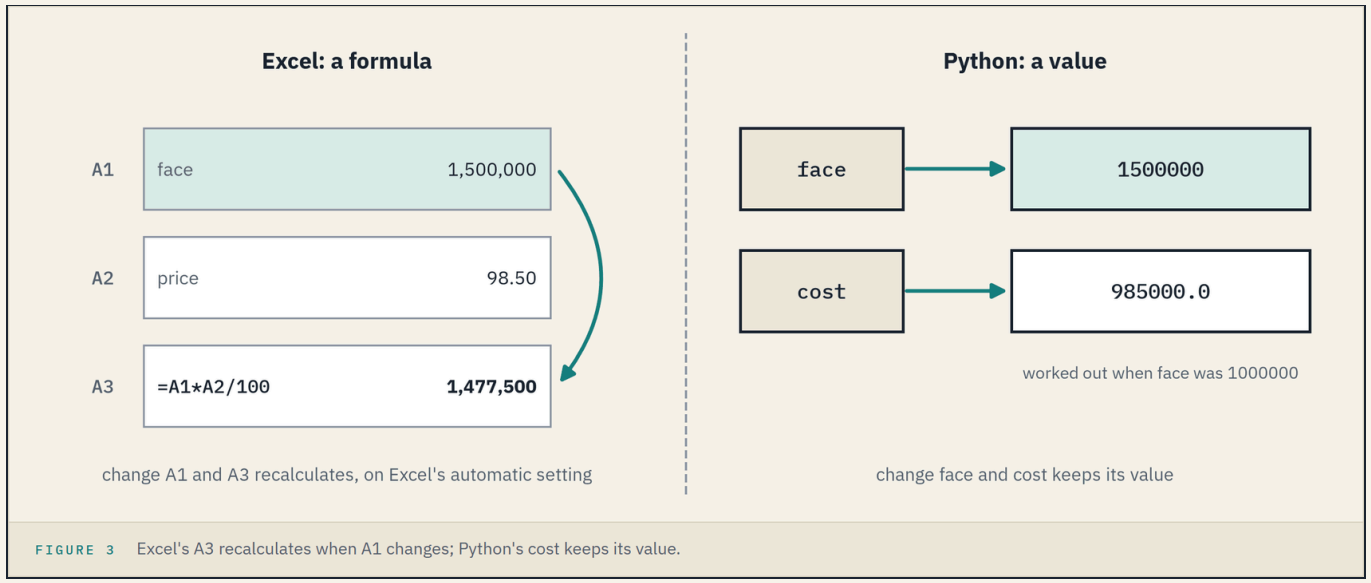


FIGURE 3 Excel's A3 recalculates when A1 changes; Python's cost keeps its value.

A NEW MARK

```
new_price = 99.25
new_value = face * new_price / 100
change = new_value - cost
print(f'Change in value: ${change:,.2f}')
```

```
Change in value: $7,500.00
```

THE DESK BUYS MORE

```
face = face + 500000
print(f'face: {face:,}   cost: ${cost:,.2f}')
```

```
face: 1,500,000   cost: $985,000.00
```

A name can take a new value; later cells use 1,500,000.

Rounding Half to Even

ROUND HALF TO EVEN

```
print(round(99.125, 2))
print(round(0.5), round(1.5), round(2.5))
```

```
99.12
0 2 2
```

ADD, THEN ROUND

```
print(f'Added first:   {added_first:.2f}')
print(f'Rounded first: {rounded_first:.2f}')
```

```
Added first:   398.00
Rounded first: 398.00
```

EXCEL SIDE BY SIDE

PRICE	PYTHON ROUND()	EXCEL =ROUND()
99.125	99.12	99.13
99.375	99.38	99.38
99.625	99.62	99.63
99.875	99.88	99.88
Total	**398.00**	**398.02**

To match a spreadsheet to the cent, check halfway cases.

1.4 Data Structures

ASK THE BOOK

```
total_face = sum(face_list)
average_coupon = sum(coupon_list) / len(coupon_list)
print(f'Bonds: {len(ticker_list)}')
print(f'Prices from {min(price_list):.2f} to {max(price_list):.2f}')
print(f'Total face: {total_face:,}')
print(f'Average coupon: {average_coupon:.3f} percent')
```

```
Bonds: 4
Prices from 95.00 to 103.25
Total face: 3,000,000
Average coupon: 5.906 percent
```

EXCEL SIDE BY SIDE

PYTHON	EXCEL
`len(price_list)`	`=COUNTA(A1:A4)`
`min(price_list), max(price_list)`	`=MIN(A1:A4), =MAX(A1:A4)`
`sum(face_list)`	`=SUM(A1:A4)`
`sum(coupon_list) / len(coupon_list)`	`=AVERAGE(A1:A4)`

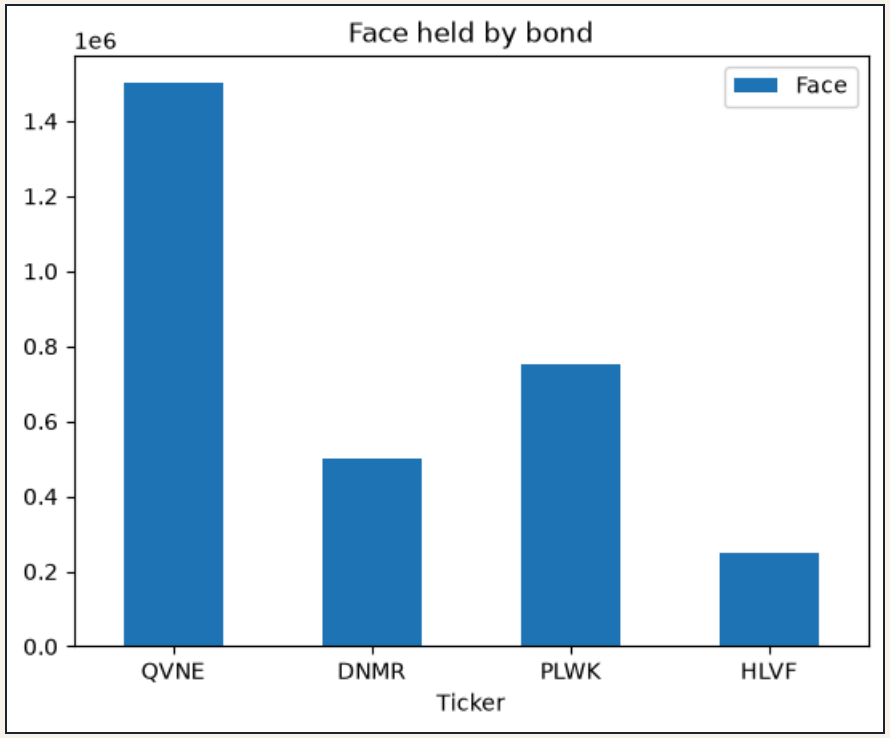
The note's third answer: 3,000,000 face, priced from 95.00 to 103.25.

Draw the Book

THE CODE

```
table = pd.DataFrame(portfolio)
```

	Ticker	Coupon	Maturity	Price	Rating	Face
0	QVNE	5.250	2033	98.50	BBB	1500000
1	DNMR	6.125	2034	103.25	BB+	500000
2	PLWK	4.750	2031	95.00	A-	750000
3	HLVF	7.500	2032	101.00	B+	250000



THREE KINDS OF BRACKETS

BRACKETS	CONTAINER	PICKS BY	CAN CHANGE
`[ ]`	list	position	yes
`( )`	tuple	position	no
`{ }`	dictionary	key	yes

QVNE is half the book: 1,500,000 of the 3,000,000 face.

1.5 Strings

CLEAN THE RATING

```
rating_text = ' bbb '
rating_clean = rating_text.strip().upper()
print(rating_clean)
```

```
BBB
```

CLEAN THE AMOUNT

```
face_text = '1,500,000'
face_number = int(face_text.replace(',', ''))
print(face_number)
```

```
1500000
```

EXCEL SIDE BY SIDE

PYTHON	EXCEL	RESULT
`rating_text.strip()`	`=TRIM(A1)`	bbb
`rating_text.strip().upper()`	`=UPPER(TRIM(A1))`	BBB
`rating_clean.lower()`	`=LOWER(A1)`	bbb
`face_text.replace(',', '')`	`=SUBSTITUTE(A1,",","")`	1500000, still text
`int(face_text.replace(',', ''))`	`=VALUE(SUBSTITUTE(A1,",",""))`	1500000, a number

Two methods in a row: `upper()` works on what `strip()` handed back.

Working With an AI Assistant

NEVER PUT INTO AN AI TOOL OR A CLOUD NOTEBOOK	FINE TO USE
Client and counterparty names	The synthetic data in this course
Positions, holdings, and trades	Public data, such as Treasury yields
Anything from your employer: files, code, emails	Code you wrote on that data
Passwords, keys, and other credentials	

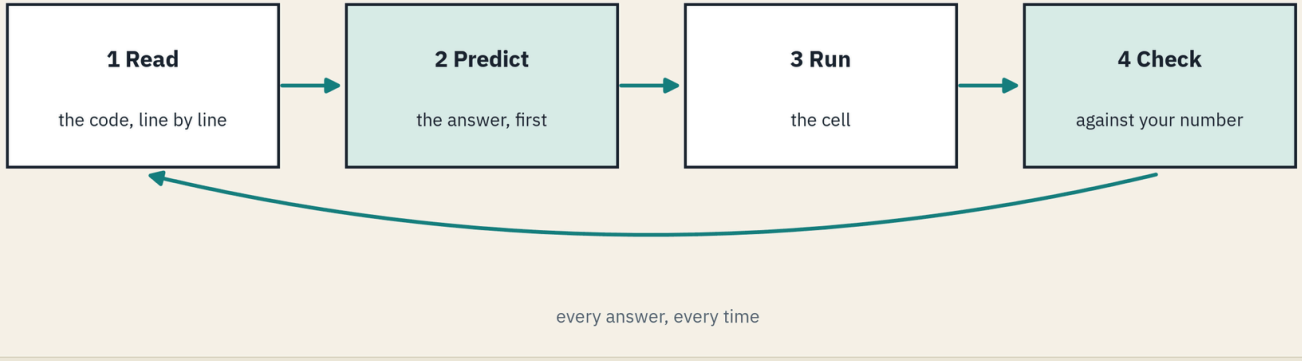


FIGURE 8 Read, predict, run, check, then round again.

Read, predict, run, check: the last exercise of every notebook practises it.

You Can Now

- Store a bond in well-named variables and know their types.
- Show a number as a desk writes it.
- Work out income, cost, whole bonds, a discount factor, and a rounding.
- Hold a portfolio in lists and dictionaries, total it, and draw it.
- Pull a string apart and clean it.
- Decide what may go into an AI tool, and check what comes back.

NEXT
WEEK 1 · DAY 1, PART 2

Python for Fixed Income: Introduction, Part 2

Decisions, loops, and functions, on the same desk of four bonds.